

CNN from Scratch with NumPy

Mathematical Derivations, Backpropagation, and the Path from Explicit Loops to Vectorized NumPy

Rousos Stamatis

Abstract

These notes document the reasoning behind a small convolutional-neural-network framework implemented from scratch with NumPy. The goal is not merely to list formulas. The goal is to make each operation understandable at three levels: first as a component-wise mathematical definition, then as an explicit loop-based implementation that mirrors the mathematics, and finally as the vectorized NumPy form used by the framework.

The emphasis is on backward propagation, especially for Dense, Conv2D, MaxPooling2D, and Batch Normalization. In each case, the central question is the same: *which quantities influence the loss, through which computational paths, and why do some gradient contributions have to be summed?* The document also covers ReLU, Flatten, Dropout, Softmax with Cross-Entropy, Sequential model composition, optimizer state, AdamW, train/evaluation modes, Early Stopping, checkpoint state, dtype choices, and numerical gradient checking.

Contents

1	Purpose and Scope	5
2	The Core Backpropagation Mental Model	5
2.1	Local derivatives and the chain rule	5
2.2	Why gradient contributions are added	5
2.3	Three levels of understanding	6
3	Notation and Shape Conventions	6
4	Dense Layer	7
4.1	Forward pass: component-wise definition	7
4.2	Forward pass: explicit loops	7
4.3	Backward pass: gradient with respect to the input	7
4.4	Backward pass: gradient with respect to the bias	8
4.5	Backward pass: gradient with respect to the weights	8
4.6	Dense backward summary	9
5	ReLU	9
5.1	Forward	9
5.2	Backward	9
6	Flatten	10
7	Conv2D	11
7.1	Forward geometry	11
7.2	Forward pass: component-wise equation	11
7.3	Forward pass: fully explicit loops	11
7.4	From windows to matrix multiplication	12

7.5	Vectorized window extraction	12
7.6	Conv2D backward overview	13
7.7	Bias gradient	13
7.8	Weight gradient: component-wise derivation	14
7.9	Weight gradient: explicit loops	14
7.10	Weight gradient: vectorization as one matrix multiplication	14
7.11	Input gradient: the first source of accumulation — filters	15
7.12	Input gradient: explicit loop for one window occurrence	16
7.13	Vectorizing the sum over filters	16
7.14	Input gradient: the second source of accumulation — overlapping windows	17
7.15	Input gradient: the completely explicit scatter-add	17
7.16	Vectorizing the scatter over output positions	17
7.17	A compact mental model for Conv2D dX	18
7.18	Removing padding	19
7.19	Conv2D backward summary	19
8	MaxPooling2D	20
8.1	Forward pass: what MaxPooling computes	20
8.2	Forward pass: explicit windows	20
8.3	Why the forward pass must remember the winner	20
8.4	Backward derivative of the max operation	21
8.5	Backward pass: fully explicit interpretation	21
8.6	Recovering local coordinates from the flattened winner index	21
8.7	Local-to-global coordinate mapping	22
8.8	Vectorizing the batch/channel loops with advanced indexing	22
8.9	Why MaxPooling backward also uses accumulation	23
8.10	MaxPooling shape and indexing summary	23
9	Batch Normalization	24
9.1	Why BatchNorm is more difficult to differentiate	24
9.2	Generic reduction convention	24
9.3	Training forward pass: computational graph	24
9.4	Running statistics and inference	25
9.5	Backward starting point	25
9.6	Gradients of β and γ	25
9.7	Backward through the affine scale	26
9.8	Backward through $\hat{x} = \tilde{x} r$	26
9.9	Backward through inverse standard deviation	26
9.10	Backward through the variance	27
9.11	Backward through centering: $\tilde{x} = x - \mu$	27
9.12	Backward through the mean	28
9.13	BatchNorm backward as a reversed computational graph	28
9.14	Why the generic Dense/Conv implementation works	29
9.15	Training versus inference backward	29
10	Dropout	30
10.1	Purpose	30
10.2	Training forward pass	30
10.3	Training backward pass	30
10.4	Evaluation behavior	30
10.5	Dropout and model lifecycle	31
11	Softmax with Cross-Entropy Loss	31

11.1	Logits and probabilities	31
11.2	Numerically stable Softmax	31
11.3	Cross-Entropy	31
11.4	Stable combined forward using log-sum-exp	32
11.5	Backward: component-wise derivation	32
11.6	Vectorized backward without constructing one-hot labels	33
11.7	Meaning of the gradient signs	33
12	Sequential Model Composition	33
12.1	The model as a composition of functions	33
12.2	Backward as repeated chain rule	33
12.3	Build lifecycle	34
12.4	Parameter references	34
12.5	Train and evaluation mode propagation	34
12.6	Persistent state and restoration	35
13	Optimizers	35
13.1	Role of an optimizer	35
13.2	SGD	35
13.3	SGD with Momentum	35
13.4	Adam: first and second moments	36
13.5	Why state is parameter-specific	36
13.6	Classical L2 regularization versus AdamW	36
13.7	AdamW update	37
13.8	Which parameters are decayable	37
13.9	In-place updates and reference stability	37
14	Early Stopping	38
14.1	Why Early Stopping is model selection, not backpropagation	38
14.2	State	38
14.3	Epoch-level logic	38
14.4	Why the best state must be copied	38
14.5	Restoring BatchNorm correctly	38
14.6	Final test evaluation	39
15	Training and Evaluation Lifecycle	39
15.1	Mini-batch training	39
15.2	Evaluation	39
15.3	Loss reporting with AdamW	40
15.4	Reproducibility and random number generators	40
16	Numerical Gradient Checking	40
16.1	Why gradient checking matters	40
16.2	Converting a tensor output into a scalar test objective	40
16.3	Checking an input gradient	41
16.4	Checking trainable parameters	41
16.5	Why float64 is useful for tests	41
16.6	Useful error measures	41
16.7	Nondifferentiable operations	41
17	State, Identity, and Mutability in the NumPy Framework	42
17.1	Why references matter	42
17.2	Identity is different from equality	42
17.3	Gradient-buffer identity	42

18 Putting the Layers Together: One Backward Pass Through the Network	42
19 A Worked Shape Trace for the CIFAR-10 Architecture	43
20 Testing Strategy	43
20.1 Behavioral/API tests	43
20.2 Mathematical tests	44
21 Design Principles Reflected by the Framework	44
21.1 Keep abstractions proportional to the project	44
21.2 Make state ownership explicit	44
21.3 Prefer explainable vectorization	44
22 Compact Reference: Forward and Backward Equations	45
22.1 Dense	45
22.2 ReLU	45
22.3 Conv2D	45
22.4 MaxPooling	45
22.5 BatchNorm	45
22.6 Dropout	45
22.7 Softmax Cross-Entropy	46
A Appendix A: Conv2D Backward as Index Mappings	47
A.1 Window occurrence to padded input	47
A.2 Flattened matrix indices	47
B Appendix B: BatchNorm Computational Graph Checklist	47
C Appendix C: Suggested Repository Documentation Split	48
D Appendix D: Final Mental Models	48
Closing Note	49

1 Purpose and Scope

This document is deliberately more detailed than a repository README. A README should let a reader understand the project quickly: what it is, how to run it, what was implemented, and what results it achieves. These notes serve a different purpose. They are the long-form derivation and implementation record behind the framework.

The framework uses NumPy only for neural-network computation. Tensors use NHWC layout:

$$X \in \mathbb{R}^{B \times H \times W \times C},$$

where B is batch size, H is height, W is width, and C is the feature/channel dimension.

The current model family is composed from layers such as

$$\text{Conv2D} \rightarrow \text{BatchNorm} \rightarrow \text{ReLU} \rightarrow \text{Conv2D} \rightarrow \text{BatchNorm} \rightarrow \text{ReLU} \rightarrow \text{MaxPool}$$

followed by additional convolutional blocks, Flatten, Dense, BatchNorm, ReLU, Dropout, and a final Dense layer that outputs logits.

Code fragments in these notes follow the implementation design and naming used in the project. Some snippets are intentionally reduced to the part relevant to the derivation, so that control-flow details do not hide the mathematics.

2 The Core Backpropagation Mental Model

2.1 Local derivatives and the chain rule

Backpropagation is easiest to reason about when a complicated forward computation is split into small operations. If

$$a \longrightarrow b \longrightarrow c \longrightarrow L,$$

then

$$\frac{\partial L}{\partial a} = \frac{\partial L}{\partial c} \frac{\partial c}{\partial b} \frac{\partial b}{\partial a}.$$

In code, the incoming gradient to a layer is often called `dout`. If the layer output is z , then

$$\text{dout} = \frac{\partial L}{\partial z}.$$

The backward method combines this upstream gradient with the layer's local derivatives to compute gradients with respect to its input and parameters.

2.2 Why gradient contributions are added

A crucial idea throughout the framework is that one variable may affect the loss through multiple computational paths. If x influences both u and v , and both influence L , then

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial u} \frac{\partial u}{\partial x} + \frac{\partial L}{\partial v} \frac{\partial v}{\partial x}.$$

The addition is not an implementation trick. It is the multivariable chain rule.

This appears repeatedly:

- A Dense weight is shared by every sample in the batch, so its gradient sums contributions across samples.
- A convolution weight is reused at every spatial window, so its gradient sums contributions across all batch/spatial positions.

- One input pixel can appear in several overlapping convolution windows, so $\partial L / \partial x$ sums contributions from all windows that contain that pixel.
- One BatchNorm input influences its normalized value directly and also influences the batch mean and variance used by all normalized values in the same feature.
- When MaxPooling windows overlap, one input may receive gradients from more than one pooled output, so scatter operations must accumulate with `+=`.

2.3 Three levels of understanding

For the difficult layers, the notes follow the same progression:

1. **Component-wise mathematics.** Write one scalar output and differentiate one scalar input or parameter.
2. **Loop-based code.** Translate the scalar equations into explicit loops. This is usually slow, but it exposes every index and every sum.
3. **Vectorized NumPy.** Identify which loops are reductions, matrix multiplications, broadcasting operations, reshapes, or scatter-adds, and replace Python loops where practical.

The vectorized implementation should always be explainable as a compact form of the explicit loop-based implementation.

3 Notation and Shape Conventions

For convolutional tensors:

$$\begin{aligned} X &: (B, H, W, C_{in}), \\ W &: (F, K_h, K_w, C_{in}), \\ b &: (F,), \\ Z &: (B, H_{out}, W_{out}, F). \end{aligned}$$

For a Dense layer:

$$\begin{aligned} X &: (B, N), \\ W &: (M, N), \\ b &: (M,), \\ Z &: (B, M). \end{aligned}$$

The upstream gradient has the same shape as the corresponding layer output. Thus if a Conv2D layer produces $Z \in \mathbb{R}^{B \times H_{out} \times W_{out} \times F}$, its backward method receives

$$D_Z = \frac{\partial L}{\partial Z} \in \mathbb{R}^{B \times H_{out} \times W_{out} \times F}.$$

Throughout the notes, s denotes stride and p denotes symmetric zero-padding unless stated otherwise.

4 Dense Layer

4.1 Forward pass: component-wise definition

For one sample $x \in \mathbb{R}^N$, one Dense neuron i computes

$$z_i = \sum_{j=0}^{N-1} w_{ij} x_j + b_i.$$

For M neurons:

$$z = Wx + b,$$

with $W \in \mathbb{R}^{M \times N}$.

For a batch $X \in \mathbb{R}^{B \times N}$, sample s is independent of every other sample:

$$z_{s,i} = \sum_j w_{ij} x_{s,j} + b_i.$$

With the project weight layout (M, N) , the vectorized batch form is

$$Z = XW^T + b,$$

where NumPy broadcasts b over the batch dimension.

4.2 Forward pass: explicit loops

```
output = np.zeros((B, M), dtype=self.dtype)

for s in range(B):
    for i in range(M):
        z = self.bias[i]
        for j in range(N):
            z += input[s, j] * self.weights[i, j]
        output[s, i] = z
```

The inner loop over j is a dot product. Replacing all such dot products at once gives the vectorized expression:

```
output = input @ self.weights.T + self.bias
```

4.3 Backward pass: gradient with respect to the input

Let

$$dout_{s,i} = \frac{\partial L}{\partial z_{s,i}}.$$

For one input feature $x_{s,j}$,

$$\frac{\partial L}{\partial x_{s,j}} = \sum_i \frac{\partial L}{\partial z_{s,i}} \frac{\partial z_{s,i}}{\partial x_{s,j}}.$$

Because

$$\frac{\partial z_{s,i}}{\partial x_{s,j}} = w_{ij},$$

we obtain

$$\frac{\partial L}{\partial x_{s,j}} = \sum_i dout_{s,i} w_{ij}.$$

The sum over i appears because the same input feature feeds every neuron in the Dense layer.

The explicit loops are:

```
din = np.zeros_like(input)

for s in range(B):
    for j in range(N):
        for i in range(M):
            din[s, j] += dout[s, i] * self.weights[i, j]
```

The inner sum is exactly matrix multiplication:

```
din = dout @ self.weights
# (B, M) @ (M, N) -> (B, N)
```

4.4 Backward pass: gradient with respect to the bias

The same bias b_i is reused for every sample:

$$z_{s,i} = \dots + b_i.$$

Therefore

$$\frac{\partial L}{\partial b_i} = \sum_s \frac{\partial L}{\partial z_{s,i}} \frac{\partial z_{s,i}}{\partial b_i} = \sum_s dout_{s,i}.$$

Thus

$$db_i = \sum_s dout_{s,i}.$$

Loop form:

```
dbias = np.zeros_like(self.bias)
for s in range(B):
    for i in range(M):
        dbias[i] += dout[s, i]
```

Vectorized form:

```
self.dbias[...] = np.sum(dout, axis=0)
```

4.5 Backward pass: gradient with respect to the weights

For one weight w_{ij} and one sample s ,

$$\left(\frac{\partial L}{\partial w_{ij}} \right)_s = \frac{\partial L}{\partial z_{s,i}} \frac{\partial z_{s,i}}{\partial w_{ij}} = dout_{s,i} x_{s,j}.$$

Because the same weight is shared by all samples, all paths contribute:

$$\frac{\partial L}{\partial w_{ij}} = \sum_s dout_{s,i} x_{s,j}.$$

Explicit loops:

```
dweights = np.zeros_like(self.weights)

for i in range(M):
    for j in range(N):
        for s in range(B):
            dweights[i, j] += dout[s, i] * input[s, j]
```

For one sample, dW_s is an outer product. For the entire batch, all outer products are summed by one matrix multiplication:

$$dW = dout^T X.$$

```
self.dweights[...] = dout.T @ self.input
# (M, B) @ (B, N) -> (M, N)
```

The matrix multiplication should not be memorized in isolation. Its element (i, j) is

$$(dout^T X)_{ij} = \sum_s dout_{s,i} x_{s,j},$$

which is exactly the component-wise derivative above.

4.6 Dense backward summary

$$\begin{aligned} dX &= dout W, \\ dW &= dout^T X, \\ db &= \sum_{s=0}^{B-1} dout_s. \end{aligned}$$

The shape logic is as important as the formulas:

- dX keeps the batch dimension because every sample owns a distinct input.
- dW has no batch dimension because there is one shared weight matrix.
- db has no batch dimension because there is one shared bias vector.

5 ReLU

5.1 Forward

ReLU is defined component-wise as

$$y_i = \max(0, x_i).$$

The vectorized forward pass is simply

```
output = np.maximum(0, input)
```

5.2 Backward

For $x_i \neq 0$,

$$\frac{\partial y_i}{\partial x_i} = \begin{cases} 1, & x_i > 0, \\ 0, & x_i < 0. \end{cases}$$

Therefore

$$\frac{\partial L}{\partial x_i} = \frac{\partial L}{\partial y_i} \mathbf{1}[x_i > 0].$$

Explicit loops:

```
din = np.empty_like(dout)
for idx in np.ndindex(input.shape):
    if input[idx] > 0:
        din[idx] = dout[idx]
    else:
        din[idx] = 0
```

Vectorized:

```
din = dout * (self.input > 0)
```

At $x = 0$ ReLU is not differentiable in the classical sense. A practical implementation chooses a convention, commonly gradient zero.

6 Flatten

Flatten changes shape but does not change numerical values. If a convolutional activation has shape

$$(B, H, W, C),$$

its forward pass may produce

$$(B, HWC).$$

```
self.input_shape = input.shape  
output = input.reshape(input.shape[0], -1)
```

The derivative of a pure reshape is the identity mapping with different indexing. Therefore backward only restores the saved shape:

```
din = dout.reshape(self.input_shape)
```

No gradient values are created, multiplied, or summed. Only their interpretation as tensor coordinates changes.

7 Conv2D

7.1 Forward geometry

The Conv2D layer receives

$$X \in \mathbb{R}^{B \times H \times W \times C_{in}}$$

and contains F filters

$$W \in \mathbb{R}^{F \times K_h \times K_w \times C_{in}}.$$

After symmetric padding p , the padded spatial dimensions are

$$H_p = H + 2p, \quad W_p = W + 2p.$$

For stride s , assuming valid integer output sizes,

$$H_{out} = \left\lfloor \frac{H_p - K_h}{s} \right\rfloor + 1, \quad W_{out} = \left\lfloor \frac{W_p - K_w}{s} \right\rfloor + 1.$$

The output has shape

$$Z \in \mathbb{R}^{B \times H_{out} \times W_{out} \times F}.$$

7.2 Forward pass: component-wise equation

For output position (b, h, w, f) , the top-left input coordinate of the corresponding receptive field is

$$r = hs, \quad q = ws$$

in padded-input coordinates. The scalar output is

$$z_{b,h,w,f} = \sum_{k_r=0}^{K_h-1} \sum_{k_c=0}^{K_w-1} \sum_{c=0}^{C_{in}-1} X_{b,hs+k_r,ws+k_c,c}^{(p)} W_{f,k_r,k_c,c} + b_f.$$

This equation is the reference point for every forward and backward derivation that follows.

7.3 Forward pass: fully explicit loops

A direct implementation follows the equation literally:

```
output = np.zeros((B, Hout, Wout, F), dtype=self.dtype)

for b in range(B):
    for h in range(Hout):
        for w in range(Wout):
            for f in range(F):
                z = self.bias[f]

                for kr in range(Kh):
                    for kc in range(Kw):
                        for c in range(Cin):
                            input_row = h * stride + kr
                            input_col = w * stride + kc

                            z += (
                                padded_input[b, input_row, input_col, c]
                                * self.weights[f, kr, kc, c]
                            )

                output[b, h, w, f] = z
```

This version is valuable because every loop corresponds to one index in the mathematical definition. It is also slow in Python because the innermost scalar operations are executed millions of times.

7.4 From windows to matrix multiplication

For each (b, h, w) , define a local input window

$$\mathcal{X}_{b,h,w} \in \mathbb{R}^{K_h \times K_w \times C_{in}}$$

whose elements are

$$\mathcal{X}_{b,h,w,k_r,k_c,c} = X_{b,hs+k_r,ws+k_c,c}^{(p)}$$

Then the forward scalar becomes

$$z_{b,h,w,f} = \sum_{k_r,k_c,c} \mathcal{X}_{b,h,w,k_r,k_c,c} W_{f,k_r,k_c,c} + b_f.$$

Now flatten the kernel-relative dimensions (k_r, k_c, c) into one index q of length

$$Q = K_h K_w C_{in}.$$

Also flatten (b, h, w) into one window index p of length

$$P = B H_{out} W_{out}.$$

This produces

$$X_{windows} \in \mathbb{R}^{P \times Q}, \quad W_{flat} \in \mathbb{R}^{F \times Q}.$$

Each row of $X_{windows}$ is one flattened convolution window. Each row of W_{flat} is one flattened filter.

The complete forward pass can therefore be written as

$$Z_{flat} = X_{windows} W_{flat}^T + b,$$

with shapes

$$(P, Q)(Q, F) \rightarrow (P, F).$$

Finally, reshape (P, F) back to (B, H_{out}, W_{out}, F) .

7.5 Vectorized window extraction

The implementation uses `np.lib.stride_tricks.sliding_window_view` to obtain all stride-1 windows as a view over the padded input, then samples the spatial window grid according to the convolution stride.

A representative form is:

```

windows = np.lib.stride_tricks.sliding_window_view(
    self.padded_input,
    axis=(1, 2),
    window_shape=(Kh, Kw),
)
# (B, Hp-Kh+1, Wp-Kw+1, C, Kh, Kw)

windows = windows[:, ::self.stride, ::self.stride, :]
# (B, Hout, Wout, C, Kh, Kw)

windows = windows.transpose(0, 1, 2, 4, 5, 3)
# (B, Hout, Wout, Kh, Kw, C)

flat_windows = windows.reshape(
    B * Hout * Wout,
    Kh * Kw * Cin,
)

```

```
# (B*Hout*Wout, Kh*Kw*Cin)

flat_weights = self.weights.reshape(F, -1)
# (F, Kh*Kw*Cin)

flat_output = flat_windows @ flat_weights.T + self.bias
output = flat_output.reshape(B, Hout, Wout, F)
```

The matrix multiplication is not changing the mathematics. It is computing all dot products between all windows and all filters at once.

7.6 Conv2D backward overview

The backward method receives

$$dout_{b,h,w,f} = \frac{\partial L}{\partial z_{b,h,w,f}}.$$

It must compute three objects:

$$\frac{\partial L}{\partial b_f}, \quad \frac{\partial L}{\partial W_{f,k_r,k_c,c}}, \quad \frac{\partial L}{\partial X_{b,i,j,c}}.$$

The three gradients have different accumulation patterns:

- db_f sums over every output generated by filter f .
- $dW_{f,k_r,k_c,c}$ sums over every batch/spatial window in which the shared weight was used.
- $dX_{b,i,j,c}$ sums over every filter and every overlapping window through which the input element affected the loss.

7.7 Bias gradient

From

$$z_{b,h,w,f} = \dots + b_f,$$

we have

$$\frac{\partial z_{b,h,w,f}}{\partial b_f} = 1.$$

The same b_f affects every batch/spatial output for filter f , so

$$\frac{\partial L}{\partial b_f} = \sum_b \sum_h \sum_w dout_{b,h,w,f}.$$

Loop form:

```
for f in range(F):
    dbias[f] = 0
    for b in range(B):
        for h in range(Hout):
            for w in range(Wout):
                dbias[f] += dout[b, h, w, f]
```

Vectorized:

```
self.dbias[...] = np.sum(dout, axis=(0, 1, 2))
# (F,)
```

7.8 Weight gradient: component-wise derivation

Consider one specific weight

$$W_{f,k_r,k_c,c}.$$

Because convolution reuses the same filter at every spatial position and for every sample, this single weight affects all outputs

$$z_{b,h,w,f}$$

for the same filter f .

For one such output,

$$\frac{\partial z_{b,h,w,f}}{\partial W_{f,k_r,k_c,c}} = X_{b,hs+kr,ws+kc,c}^{(p)}.$$

Therefore the multivariable chain rule gives

$$\frac{\partial L}{\partial W_{f,k_r,k_c,c}} = \sum_b \sum_h \sum_w \frac{\partial L}{\partial z_{b,h,w,f}} X_{b,hs+kr,ws+kc,c}^{(p)}.$$

Using `dout`,

$$dW_{f,k_r,k_c,c} = \sum_{b,h,w} dout_{b,h,w,f} \mathcal{X}_{b,h,w,k_r,k_c,c}.$$

The sum exists because the same parameter participates in many computational paths. Each use of the weight contributes one term to its final gradient.

7.9 Weight gradient: explicit loops

```
dweights = np.zeros_like(self.weights)

for f in range(F):
    for kr in range(Kh):
        for kc in range(Kw):
            for c in range(Cin):
                for b in range(B):
                    for h in range(Hout):
                        for w in range(Wout):
                            input_row = h * stride + kr
                            input_col = w * stride + kc

                            dweights[f, kr, kc, c] += (
                                dout[b, h, w, f]
                                * padded_input[b, input_row, input_col, c]
                            )
```

This is a direct translation of the derivative. The expensive part is the reduction over all (b, h, w) for every weight.

7.10 Weight gradient: vectorization as one matrix multiplication

Flatten

$$dout \rightarrow D \in \mathbb{R}^{P \times F}, \quad P = BH_{out}W_{out}.$$

The implementation uses

```
flat_dout = np.reshape(
    dout,
    shape=(batch_size * Hout * Wout, -1),
)
# (B*Hout*Wout, F)
```

Recall

$$X_{windows} \in \mathbb{R}^{P \times Q}, \quad Q = K_h K_w C_{in}.$$

Then

$$D^T X_{windows} \in \mathbb{R}^{F \times Q}.$$

Look at one element (f, q) :

$$(D^T X_{windows})_{f,q} = \sum_{p=0}^{P-1} D_{p,f} X_{windows,p,q}.$$

The flattened index p is exactly one (b, h, w) position, while q is exactly one (k_r, k_c, c) position. Therefore

$$(D^T X_{windows})_{f,q} = \sum_{b,h,w} dout_{b,h,w,f} \mathcal{X}_{b,h,w,k_r,k_c,c},$$

which is the component-wise gradient derived above.

Hence:

```
dL_dw_flat = flat_dout.T @ flat_windows
# (F, B*Hout*Wout) @ (B*Hout*Wout, Kh*Kw*C)
# -> (F, Kh*Kw*C)

self.dweights[...] = np.reshape(
    dL_dw_flat,
    shape=(-1, Kh, Kw, self.in_channels),
)
# (F, Kh, Kw, C)
```

A useful interpretation is:

Row f of `flat_dout.T` contains all upstream gradients associated with filter f . Column q of `flat_windows` contains the same kernel-relative input position $q = (k_r, k_c, c)$ from every convolution window. Their dot product sums exactly the gradient contributions to weight (f, k_r, k_c, c) from every place where that weight was used.

7.11 Input gradient: the first source of accumulation — filters

The input gradient is more subtle because an input element can influence many outputs.

Start with one occurrence of one input element inside one specific window. Let

$$\mathcal{X}_{b,h,w,k_r,k_c,c}$$

be the element at local kernel position (k_r, k_c, c) inside window (b, h, w) .

Within this fixed window, that element contributes to the output of every filter:

$$z_{b,h,w,f} = \sum_{k_r,k_c,c} \mathcal{X}_{b,h,w,k_r,k_c,c} W_{f,k_r,k_c,c} + b_f.$$

For one filter,

$$\frac{\partial z_{b,h,w,f}}{\partial \mathcal{X}_{b,h,w,k_r,k_c,c}} = W_{f,k_r,k_c,c}.$$

Since the same window element affects all F filters, its gradient is

$$\frac{\partial L}{\partial \mathcal{X}_{b,h,w,k_r,k_c,c}} = \sum_f dout_{b,h,w,f} W_{f,k_r,k_c,c}.$$

This is the first accumulation in dX : **sum over filters**.

7.12 Input gradient: explicit loop for one window occurrence

If we temporarily treat every occurrence inside every window as a separate variable, we could compute

```
dwindows = np.zeros((B, Hout, Wout, Kh, Kw, Cin), dtype=self.dtype)

for b in range(B):
    for h in range(Hout):
        for w in range(Wout):
            for kr in range(Kh):
                for kc in range(Kw):
                    for c in range(Cin):
                        for f in range(F):
                            dwindows[b, h, w, kr, kc, c] += (
                                dout[b, h, w, f]
                                * self.weights[f, kr, kc, c]
                            )
```

At this stage, `dwindows` is *not yet* dL/dX . It contains gradients per *window occurrence*. If one original input pixel appears in several overlapping windows, it appears several times in `dwindows`.

7.13 Vectorizing the sum over filters

Flatten the filter weights:

```
flat_weights = np.reshape(
    self.weights,
    shape=(self.filters, -1),
)
# (F, Kh*Kw*C)
```

Then

```
dwindows_flat = flat_dout @ flat_weights
# (B*Hout*Wout, F) @ (F, Kh*Kw*C)
# -> (B*Hout*Wout, Kh*Kw*C)
```

For one output element (p, q) of this multiplication,

$$(dwindows_{flat})_{p,q} = \sum_f flat_dout_{p,f} flat_weights_{f,q}.$$

After mapping $p \leftrightarrow (b, h, w)$ and $q \leftrightarrow (k_r, k_c, c)$, this is exactly

$$\sum_f dout_{b,h,w,f} W_{f,k_r,k_c,c}.$$

Reshaping restores the meaningful window coordinates:

```
dwindows = np.reshape(
    dwindows_flat,
    shape=(batch_size, Hout, Wout, Kh, Kw, self.in_channels),
)
# (B, Hout, Wout, Kh, Kw, C)
```

The useful mental model is:

`dwindows` contains gradients per window occurrence. Its indices still describe *which convolution window* and *which local position inside that window*, not yet the unique coordinates of the original input tensor.

7.14 Input gradient: the second source of accumulation — overlapping windows

The relationship between a window element and the padded input is

$$\mathcal{X}_{b,h,w,k_r,k_c,c} = X_{b,hs+kr,ws+kc,c}^{(p)}.$$

Different tuples (h, w, k_r, k_c) can refer to the same padded-input position.

For example with stride $s = 1$:

$$(h = 0, w = 0, k_r = 0, k_c = 1)$$

and

$$(h = 0, w = 1, k_r = 0, k_c = 0)$$

both refer to padded-input column 1 because

$$0 \cdot 1 + 1 = 1, \quad 1 \cdot 1 + 0 = 1.$$

Therefore the corresponding entries in `dwindows` are two different computational-path contributions to the same original input element and must be added.

This is the second accumulation in dX : **sum over overlapping window occurrences**.

7.15 Input gradient: the completely explicit scatter-add

The most transparent implementation is:

```
dL_dx_padded = np.zeros_like(self.padded_input)

for b in range(B):
    for h in range(Hout):
        for w in range(Wout):
            for kr in range(Kh):
                for kc in range(Kw):
                    for c in range(Cin):
                        input_row = h * stride + kr
                        input_col = w * stride + kc

                        dL_dx_padded[b, input_row, input_col, c] += (
                            dwindows[b, h, w, kr, kc, c]
                        )
```

The `+=` is essential. Assignment with `=` would overwrite earlier paths whenever windows overlap.

7.16 Vectorizing the scatter over output positions

The implementation keeps only the two loops over kernel position:

```
for kr in range(Kh):
    for kc in range(Kw):
        dL_dx_padded[
            :,
            kr:kr + Hout * self.stride:self.stride,
            kc:kc + Wout * self.stride:self.stride,
            :,
        ] += dwindows[:, :, :, kr, kc, :]
```

This line becomes much easier to understand by expanding it back into the loops it replaces:

```

for kr in range(Kh):
    for kc in range(Kw):
        for h in range(Hout):
            for w in range(Wout):
                dL_dx_padded[
                    :,
                    h * stride + kr,
                    w * stride + kc,
                    :,
                ] += dwindows[:, h, w, kr, kc, :]

```

For a fixed (k_r, k_c) ,

```

dwindows[:, :, :, kr, kc, :]

```

has shape $(B, H_{out}, W_{out}, C_{in})$. It contains the gradient of the local (k_r, k_c) window position for every convolution window and every channel.

The left-hand slice

```

dL_dx_padded[
    :,
    kr:kr + Hout * stride:stride,
    kc:kc + Wout * stride:stride,
    :,
]

```

also has shape $(B, H_{out}, W_{out}, C_{in})$. Its internal index (h, w) refers to padded-input coordinate

$$(hs + k_r, ws + k_c).$$

Thus the slice performs the mapping from **window coordinates** to **input-pixel coordinates** for all output positions at once.

The step in the slice is the stride because, as h increases by one, the start of the next convolution window moves by s input rows. The selected row indices are

$$k_r, k_r + s, k_r + 2s, \dots,$$

which are exactly $hs + k_r$. The same reasoning applies to columns.

7.17 A compact mental model for Conv2D dX

The complete input-gradient computation can be understood in two stages:

1. **Per window occurrence:** `flat_dout @ flat_weights` sums over filters and computes the gradient of every element of every extracted window.
2. **Back to the real input:** scatter-add maps each window occurrence back to the padded-input coordinate from which it came and adds contributions whenever multiple occurrences correspond to the same original input element.

In one line:

```

dwindows = gradients per window occurrence
           ↓ scatter-add
dL/dx = gradients per actual input element

```

A general component-wise expression for one padded-input element is

$$\frac{\partial L}{\partial X_{b,i,j,c}^{(p)}} = \sum_{\substack{h,w,k_r,k_c \\ i=hs+k_r \\ j=ws+k_c}} \sum_f dout_{b,h,w,f} W_{f,k_r,k_c,c}.$$

The conditions underneath the outer sum mean: include every window/kernel-position pair that maps to the input coordinate (i, j) .

7.18 Removing padding

The scatter is performed into the padded-input gradient because the forward windows were extracted from the padded input. The final layer input gradient is obtained by cropping back to the original input region.

Conceptually, if the original spatial shape is (H, W) , the safest statement is

```
din = dL_dx_padded[:, p:p + H, p:p + W, :]
```

which also handles $p = 0$ naturally.

7.19 Conv2D backward summary

The backward pass contains three different reductions:

$$dW_{f,k_r,k_c,c} = \sum_{b,h,w} dout_{b,h,w,f} X_{b,hs+k_r,ws+k_c,c}^{(p)},$$

$$dX_{b,i,j,c}^{(p)} = \sum_{\text{windows containing } (i,j,c)} \sum_f dout \cdot W.$$

The vectorized implementation mirrors these sums:

- `np.sum(..., axis=(0,1,2))` for db ,
- `flat_dout.T @ flat_windows` for dW ,
- `flat_dout @ flat_weights` followed by scatter-add for dX .

8 MaxPooling2D

8.1 Forward pass: what MaxPooling computes

For a pooling window of size (K_h, K_w) , MaxPooling acts independently for each sample and channel. At output location (h, w) and channel c ,

$$y_{b,h,w,c} = \max_{\substack{0 \leq k_r < K_h \\ 0 \leq k_c < K_w}} X_{b,hs+k_r,ws+k_c,c}.$$

Unlike convolution, MaxPooling has no trainable parameters. Its backward pass is nevertheless nontrivial because only the input element that won the maximum operation receives the upstream gradient.

8.2 Forward pass: explicit windows

For one output spatial location, the loop-based forward pass may extract

```

window = input[
    :,
    input_row:input_row + Kh,
    input_col:input_col + Kw,
    :,
]
# (B, Kh, Kw, C)

```

For fixed (b, c) , `window[b, :, :, c]` is a $K_h \times K_w$ matrix. The maximum over the two spatial axes is

```

max_element = np.max(window, axis=(1, 2))
# (B, C)

```

and can be written into one output spatial position:

```

output[:, output_row, output_col, :] = max_element

```

8.3 Why the forward pass must remember the winner

The backward pass needs to know which local input coordinate produced each maximum. The maximum value alone is not enough. Therefore the spatial part of the window is flattened:

```

flat_window = np.reshape(window, (B, Kh * Kw, C))
# axis 0 -> batch
# axis 1 -> flattened local spatial position
# axis 2 -> channel

```

Then

```

max_element_idx = np.argmax(flat_window, axis=1)
# (B, C)

```

stores one local flattened index for each (b, c) pair.

For a 2×2 window, the local index map is

```

0 1
2 3

```

If the winner is local row 1, column 0, the stored index is 2. These are *local window coordinates*; they are not coordinates in the full input tensor.

8.4 Backward derivative of the max operation

For one scalar pooled output

$$y = \max(x_1, \dots, x_n),$$

assuming a unique maximum at index m ,

$$\frac{\partial y}{\partial x_j} = \begin{cases} 1, & j = m, \\ 0, & j \neq m. \end{cases}$$

Therefore

$$\frac{\partial L}{\partial x_j} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial x_j} = \begin{cases} dout, & j = m, \\ 0, & j \neq m. \end{cases}$$

This is the entire mathematical idea behind MaxPooling backward: route each upstream gradient to the winner from the forward pass.

If multiple elements are exactly tied for the maximum, the mathematical max function is nondifferentiable at that point. An `argmax`-based implementation chooses one winner according to NumPy's tie-breaking behavior and sends the gradient there. This convention is deterministic and sufficient for the framework.

8.5 Backward pass: fully explicit interpretation

The most direct code would be:

```
din = np.zeros_like(input)

for output_row in range(Hout):
    for output_col in range(Wout):
        input_row = output_row * stride
        input_col = output_col * stride

        for b in range(B):
            for c in range(C):
                flat_idx = self.max_element_idx[
                    b, output_row, output_col, c
                ]

                window_row, window_col = np.unravel_index(
                    flat_idx,
                    (Kh, Kw),
                )

                global_row = input_row + window_row
                global_col = input_col + window_col

                din[b, global_row, global_col, c] += (
                    dout[b, output_row, output_col, c]
                )
```

This explicit version is the reference implementation conceptually. The vectorized code performs the same coordinate construction without the inner loops over batch and channels.

8.6 Recovering local coordinates from the flattened winner index

At one output position,

```
flat_window_idx = self.max_element_idx[
    :, output_row, output_col, :
]
# (B, C)
```

contains a local flattened winner index for every image and channel.

Calling

```
window_rows, window_cols = np.unravel_index(
    flat_window_idx,
    (Kh, Kw),
)
```

returns two arrays of shape (B, C) . For each pair (b, c) ,

$$(window_rows_{b,c}, window_cols_{b,c})$$

is the local two-dimensional position of the maximum.

For a 2×3 window,

$$\begin{pmatrix} 0 & 1 & 2 \\ 3 & 4 & 5 \end{pmatrix}$$

so flat index 4 maps to local coordinate $(1, 1)$.

8.7 Local-to-global coordinate mapping

The pooling window for output location (h, w) starts at

$$input_row = hs, \quad input_col = ws.$$

Therefore

$$global_row = hs + window_row, \quad global_col = ws + window_col.$$

In code:

```
global_rows = input_row + window_rows
# (B, C)

global_cols = input_col + window_cols
# (B, C)
```

At this point the destination coordinate for every (b, c) pair is known.

8.8 Vectorizing the batch/channel loops with advanced indexing

Create batch and channel index arrays:

```
batch_idx = np.arange(B)[:, np.newaxis]    # (B, 1)
channel_idx = np.arange(C)[np.newaxis, :]   # (1, C)
```

Broadcasting makes them conceptually align with the (B, C) arrays `global_rows` and `global_cols`. The advanced-index expression

```
din[batch_idx, global_rows, global_cols, channel_idx]
```

selects exactly one input element for every (b, c) pair:

$$(b, \text{global_rows}_{b,c}, \text{global_cols}_{b,c}, c).$$

The upstream gradients at the current output spatial location are

```
dout[:, output_row, output_col, :]
# (B, C)
```

so the vectorized routing operation is

```
din[
    batch_idx,
    global_rows,
    global_cols,
    channel_idx,
] += dout[:, output_row, output_col, :]
```

This line replaces only the loops over b and c . The outer loops over output spatial positions may remain because the purpose is clarity and because they are much smaller than a fully scalar implementation.

8.9 Why MaxPooling backward also uses accumulation

If pooling windows overlap, the same input element may be the maximum for more than one pooling window. Then multiple output values depend on that same input value, and the chain rule requires their gradient paths to be added.

Thus

```
+=
```

is correct, whereas plain assignment could erase an earlier contribution.

The mental model is:

During forward, each pooled output remembers which local input element won. During backward, each upstream gradient is routed back to that winner. If several pooled outputs route to the same input, the contributions are summed.

8.10 MaxPooling shape and indexing summary

For a fixed output spatial location:

```
stored flat winner indices : (B, C),
local winner rows : (B, C),
local winner cols : (B, C),
global winner rows : (B, C),
global winner cols : (B, C),
upstream gradients : (B, C).
```

The advanced-index assignment performs the same (b, c) -wise operation that the explicit nested loops would perform.

9 Batch Normalization

9.1 Why BatchNorm is more difficult to differentiate

Batch Normalization is one of the most useful layers for learning the computational-graph view of backpropagation. A Dense or Conv output can often be differentiated directly from one compact equation. BatchNorm is different because each input feature value contributes to the batch statistics that are then reused to normalize many values.

For one feature/channel, an input x_i affects the loss through several paths:

1. directly through its centered value $x_i - \mu$,
2. through the mean μ , which affects every centered value,
3. through the variance, which depends on all centered values,
4. through the inverse standard deviation, which scales every normalized value.

The safest derivation is therefore to write BatchNorm as a sequence of small forward operations and reverse them one by one.

9.2 Generic reduction convention

The project uses the last tensor dimension as the feature/channel dimension. Therefore BatchNorm can use the same implementation for Dense and Conv inputs.

For Dense:

$$X : (B, D),$$

so statistics are reduced over axis 0.

For Conv:

$$X : (B, H, W, C),$$

so statistics are reduced over axes (0, 1, 2) independently for each channel.

This can be represented as

```
reduction_axis = tuple(range(input.ndim - 1))
N = np.prod(input.shape[:reduction_axis])
```

For the derivation, all reduced dimensions are collapsed conceptually into one index $i \in \{1, \dots, N\}$. The feature/channel index is denoted by c .

9.3 Training forward pass: computational graph

For each feature c :

$$\mu_c = \frac{1}{N} \sum_i x_{i,c}, \tag{1}$$

$$\tilde{x}_{i,c} = x_{i,c} - \mu_c, \tag{2}$$

$$\sigma_c^2 = \frac{1}{N} \sum_i \tilde{x}_{i,c}^2, \tag{3}$$

$$r_c = (\sigma_c^2 + \epsilon)^{-1/2}, \tag{4}$$

$$\hat{x}_{i,c} = \tilde{x}_{i,c} r_c, \tag{5}$$

$$y_{i,c} = \gamma_c \hat{x}_{i,c} + \beta_c. \tag{6}$$

The trainable parameters are

$$\gamma, \beta \in \mathbb{R}^C.$$

The batch statistics μ and σ^2 are not trainable parameters.

A vectorized forward implementation follows these operations:

```
mean = np.mean(input, axis=reduction_axis, keepdims=True)
centered = input - mean
variance = np.mean(centered ** 2, axis=reduction_axis, keepdims=True)
inv_std = 1.0 / np.sqrt(variance + self.epsilon)
normalized = centered * inv_std
output = self.gamma * normalized + self.beta
```

Because the last dimension is the feature dimension, **gamma** and **beta** broadcast over every reduced axis.

9.4 Running statistics and inference

During training, BatchNorm also updates persistent running statistics. With momentum m under the convention used by the project, the update is conceptually of the form

$$\begin{aligned} running_mean &\leftarrow m running_mean + (1 - m) batch_mean, \\ running_variance &\leftarrow m running_variance + (1 - m) batch_variance. \end{aligned}$$

During inference, the current batch must not define the normalization statistics. Instead,

$$\hat{x} = \frac{x - running_mean}{\sqrt{running_variance + \epsilon}}, \quad y = \gamma \hat{x} + \beta.$$

This is why BatchNorm contains both trainable state (γ, β) and persistent non-trainable state $(running_mean, running_variance)$. A complete model restore intended for inference must restore all four.

9.5 Backward starting point

Let

$$g_{i,c} = \frac{\partial L}{\partial y_{i,c}} = dout_{i,c}.$$

We now reverse the small forward operations rather than jumping to a memorized compact BatchNorm formula.

9.6 Gradients of β and γ

From

$$y_{i,c} = \gamma_c \hat{x}_{i,c} + \beta_c,$$

we have

$$\frac{\partial y_{i,c}}{\partial \beta_c} = 1, \quad \frac{\partial y_{i,c}}{\partial \gamma_c} = \hat{x}_{i,c}.$$

Since the same β_c and γ_c are shared over all N normalized positions,

$$d\beta_c = \sum_i g_{i,c}$$

and

$$d\gamma_c = \sum_i g_{i,c} \hat{x}_{i,c}.$$

Vectorized:

```
self.dbeta[...] = np.sum(dout, axis=reduction_axis)
self.dgamma[...] = np.sum(dout * self.normalized, axis=reduction_axis)
```

The in-place assignment with [...] matters because optimizers may already hold references to the gradient arrays returned by `parameters()`.

9.7 Backward through the affine scale

The normalized activation receives

$$d\hat{x}_{i,c} = g_{i,c}\gamma_c.$$

Vectorized:

```
dnormalized = dout * self.gamma
```

9.8 Backward through $\hat{x} = \tilde{x} r$

The normalized value is a product:

$$\hat{x}_{i,c} = \tilde{x}_{i,c} r_c.$$

Therefore it creates two gradient paths.

First, the direct path to the centered value:

$$(d\tilde{x}_{i,c})_{direct} = d\hat{x}_{i,c} r_c.$$

Second, the path to the shared inverse standard deviation:

$$\frac{\partial \hat{x}_{i,c}}{\partial r_c} = \tilde{x}_{i,c}.$$

Since the same r_c is used for all i ,

$$dr_c = \sum_i d\hat{x}_{i,c} \tilde{x}_{i,c}.$$

Vectorized:

```
dcentered_direct = dnormalized * self.inv_std

dinv_std = np.sum(
    dnormalized * self.centered,
    axis=reduction_axis,
    keepdims=True,
)
```

9.9 Backward through inverse standard deviation

The forward operation is

$$r_c = (\sigma_c^2 + \epsilon)^{-1/2}.$$

Its derivative is

$$\frac{\partial r_c}{\partial \sigma_c^2} = -\frac{1}{2}(\sigma_c^2 + \epsilon)^{-3/2}.$$

Therefore

$$d\sigma_c^2 = dr_c \left[-\frac{1}{2}(\sigma_c^2 + \epsilon)^{-3/2} \right].$$

A code form is:

```

dvariance = (
    dinv_std
    * (-0.5)
    * (self.variance + self.epsilon) ** (-1.5)
)

```

9.10 Backward through the variance

The variance is

$$\sigma_c^2 = \frac{1}{N} \sum_i \tilde{x}_{i,c}^2.$$

For one centered value,

$$\frac{\partial \sigma_c^2}{\partial \tilde{x}_{i,c}} = \frac{2}{N} \tilde{x}_{i,c}.$$

Therefore the variance path contributes

$$(d\tilde{x}_{i,c})_{var} = d\sigma_c^2 \frac{2}{N} \tilde{x}_{i,c}.$$

Vectorized:

```

dcentered_from_variance = (
    dvariance * (2.0 / N) * self.centered
)

```

Now the same centered variable has received gradients through two distinct computational paths, so they must be added:

```

dcentered = dcentered_direct + dcentered_from_variance

```

This addition is an important example of gradient accumulation that comes directly from the graph structure.

9.11 Backward through centering: $\tilde{x} = x - \mu$

Each centered value satisfies

$$\tilde{x}_{i,c} = x_{i,c} - \mu_c.$$

The direct derivative to its own input is

$$\frac{\partial \tilde{x}_{i,c}}{\partial x_{i,c}} = 1.$$

The derivative with respect to the shared mean is

$$\frac{\partial \tilde{x}_{i,c}}{\partial \mu_c} = -1.$$

Therefore all centered values contribute to the mean gradient:

$$d\mu_c = - \sum_i d\tilde{x}_{i,c}.$$

Vectorized:

```

dmean = -np.sum(
    dcentered,
    axis=reduction_axis,
    keepdims=True,
)

```

9.12 Backward through the mean

The mean is

$$\mu_c = \frac{1}{N} \sum_i x_{i,c}.$$

For every i ,

$$\frac{\partial \mu_c}{\partial x_{i,c}} = \frac{1}{N}.$$

Hence the mean path contributes the same amount to every input:

$$(dx_{i,c})_{mean} = d\mu_c \frac{1}{N}.$$

The input also receives the direct centering gradient $d\tilde{x}_{i,c}$. Adding both paths gives

$$dx_{i,c} = d\tilde{x}_{i,c} + \frac{d\mu_c}{N}.$$

Vectorized:

```
din = dcentered + dmean / N
```

9.13 BatchNorm backward as a reversed computational graph

The full backward can be read in reverse order from the forward graph:

```
# y = gamma * normalized + beta
self.dbeta[...] = np.sum(dout, axis=reduction_axis)
self.dgamma[...] = np.sum(
    dout * self.normalized,
    axis=reduction_axis,
)
dnormalized = dout * self.gamma

# normalized = centered * inv_std
dcentered_direct = dnormalized * self.inv_std
dinv_std = np.sum(
    dnormalized * self.centered,
    axis=reduction_axis,
    keepdims=True,
)

# inv_std = (variance + eps)^(-1/2)
dvariance = (
    dinv_std
    * (-0.5)
    * (self.variance + self.epsilon) ** (-1.5)
)

# variance = mean(centered ** 2)
dcentered_from_variance = (
    dvariance * (2.0 / N) * self.centered
)
dcentered = dcentered_direct + dcentered_from_variance

# centered = input - mean
dmean = -np.sum(
    dcentered,
```

```

    axis=reduction_axis,
    keepdims=True,
)

# mean = mean(input)
din = dcentered + dmean / N

```

This form is intentionally longer than a compact closed-form BatchNorm derivative. It mirrors the actual forward graph and makes every gradient path visible.

9.14 Why the generic Dense/Conv implementation works

The equations above use an abstract reduced index i . For Dense input (B, D) , i is the batch index. For Conv input (B, H, W, C) , i represents the combined tuple (b, h, w) . The feature dimension remains last in both cases.

Thus operations such as

```
np.sum(..., axis=reduction_axis, keepdims=True)
```

perform exactly the mathematical sum over i while preserving a broadcastable last-dimension feature tensor.

9.15 Training versus inference backward

Backpropagation is part of training. During normal inference, BatchNorm uses running statistics and no backward pass is performed. The important lifecycle distinction is therefore:

- **training forward:** use batch statistics, update running statistics, cache intermediates needed by backward;
- **evaluation forward:** use running statistics and do not update them;
- **backward:** used only for a training forward whose cached graph is valid.

10 Dropout

10.1 Purpose

Dropout is a training-time regularization technique. It randomly removes activations so that the network cannot rely too strongly on particular activation paths. The project uses **inverted dropout**, which performs the scaling during training so that evaluation requires no rescaling.

Let the drop probability be p and the keep probability be

$$q = 1 - p.$$

10.2 Training forward pass

For each activation x_i , sample a Bernoulli mask

$$m_i \sim \text{Bernoulli}(q).$$

In inverted dropout,

$$y_i = x_i \frac{m_i}{q}.$$

Thus an activation is either zero or scaled by $1/q$.

The expectation is preserved:

$$\mathbb{E}[y_i] = x_i \frac{\mathbb{E}[m_i]}{q} = x_i \frac{q}{q} = x_i.$$

A typical vectorized implementation is:

```
keep_prob = 1.0 - self.drop_prob
self.mask = (self.rng.random(input.shape) < keep_prob) / keep_prob
output = input * self.mask
```

10.3 Training backward pass

During a particular forward pass, the mask is a fixed tensor. Therefore

$$\frac{\partial y_i}{\partial x_i} = \frac{m_i}{q}.$$

The input gradient is

$$dx_i = dout_i \frac{m_i}{q}.$$

If the forward implementation stores the already-scaled mask, the backward is simply

```
din = dout * self.mask
```

Dropped activations receive zero gradient because their forward computational path was removed for that training step.

10.4 Evaluation behavior

Because inverted dropout preserves the activation expectation during training, evaluation is the identity function:

$$y = x.$$

Thus

```
if not self.training:
    return input
```

No random mask should be generated during evaluation.

10.5 Dropout and model lifecycle

Dropout is one reason a model-level `train()` / `eval()` API is useful. The layer's numerical operation depends on mode even though it has no trainable parameters.

A useful behavioral unit test is:

After switching the model or layer to evaluation mode, Dropout output must be exactly equal to the input.

11 Softmax with Cross-Entropy Loss

11.1 Logits and probabilities

The final Dense layer produces logits

$$Z \in \mathbb{R}^{B \times M},$$

where M is the number of classes. For one sample, Softmax defines

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}.$$

For a batch, Softmax is applied independently to each row. Samples do not interact through the class normalization.

11.2 Numerically stable Softmax

Direct exponentiation can overflow for large logits. For each sample b , let

$$m_b = \max_j z_{b,j}.$$

Subtracting m_b does not change Softmax probabilities:

$$p_{b,i} = \frac{e^{z_{b,i} - m_b}}{\sum_j e^{z_{b,j} - m_b}}.$$

The largest shifted logit is zero, so no exponent is positive.

Vectorized:

```
max_logits = np.max(logits, axis=1, keepdims=True)
shifted = logits - max_logits
exp_shifted = np.exp(shifted)
probabilities = exp_shifted / np.sum(
    exp_shifted,
    axis=1,
    keepdims=True,
)
```

11.3 Cross-Entropy

For sample b with integer target class y_b ,

$$L_b = -\log p_{b,y_b}.$$

The batch loss is the mean

$$L = \frac{1}{B} \sum_b L_b.$$

11.4 Stable combined forward using log-sum-exp

Computing $-\log p_{b,y_b}$ after explicitly forming a very small probability can underflow. A more stable expression works directly with logits.

Starting from

$$L_b = -\log \left(\frac{e^{z_{b,y_b}}}{\sum_j e^{z_{b,j}}} \right),$$

we obtain

$$L_b = -z_{b,y_b} + \log \sum_j e^{z_{b,j}}.$$

Using $m_b = \max_j z_{b,j}$,

$$\log \sum_j e^{z_{b,j}} = m_b + \log \sum_j e^{z_{b,j}-m_b}.$$

Therefore

$$L_b = -z_{b,y_b} + m_b + \log \sum_j e^{z_{b,j}-m_b}.$$

At least one shifted exponent is exactly $e^0 = 1$, so the logarithm receives a positive finite sum even if very small-class exponentials underflow to zero.

11.5 Backward: component-wise derivation

We want

$$\frac{\partial L}{\partial z_{b,i}}.$$

Since the batch loss is a mean and sample b affects only L_b ,

$$\frac{\partial L}{\partial z_{b,i}} = \frac{1}{B} \frac{\partial L_b}{\partial z_{b,i}}.$$

Let $k = y_b$ be the correct class. The sample loss is

$$L_b = -\log p_{b,k}.$$

Thus

$$\frac{\partial L_b}{\partial p_{b,k}} = -\frac{1}{p_{b,k}}.$$

The Softmax derivative has two cases:

$$\frac{\partial p_k}{\partial z_i} = \begin{cases} p_k(1 - p_k), & i = k, \\ -p_k p_i, & i \neq k. \end{cases}$$

For an incorrect class $i \neq k$,

$$\frac{\partial L}{\partial z_{b,i}} = \frac{1}{B} \left(-\frac{1}{p_{b,k}} \right) (-p_{b,k} p_{b,i}) = \boxed{\frac{p_{b,i}}{B}}.$$

For the correct class $i = k$,

$$\frac{\partial L}{\partial z_{b,k}} = \frac{1}{B} \left(-\frac{1}{p_{b,k}} \right) p_{b,k}(1 - p_{b,k}) = \boxed{\frac{p_{b,k} - 1}{B}}.$$

Combining both cases, if Y is the conceptual one-hot label matrix,

$$\boxed{\frac{\partial L}{\partial Z} = \frac{P - Y}{B}}.$$

11.6 Vectorized backward without constructing one-hot labels

Integer labels already tell us which entry in each row should subtract one:

```
dlogits = self.probabilities.copy()
dlogits[np.arange(B), y] -= 1.0
dlogits /= B
```

The output has shape (B, M) and can be passed directly to the final Dense layer's backward method.

11.7 Meaning of the gradient signs

For an incorrect class,

$$\frac{\partial L}{\partial z_i} = p_i / B > 0.$$

Gradient descent subtracts a positive quantity, tending to decrease the incorrect logit.

For the correct class,

$$\frac{\partial L}{\partial z_k} = (p_k - 1) / B < 0,$$

so gradient descent subtracts a negative quantity, tending to increase the correct logit.

12 Sequential Model Composition

12.1 The model as a composition of functions

A Sequential model with layers $f_1, \dots, f_n$ represents

$$F(x) = f_n(f_{n-1}(\dots f_2(f_1(x)) \dots)).$$

The forward pass applies layers from first to last:

```
def forward(self, input):
    x = input
    for layer in self.layers:
        x = layer.forward(x)
    return x
```

Each layer caches only the information it needs for its own backward pass.

12.2 Backward as repeated chain rule

If

$$x_0 \xrightarrow{f_1} x_1 \xrightarrow{f_2} \dots \xrightarrow{f_n} x_n \rightarrow L,$$

then backpropagation starts from dL/dx_n and applies the local Jacobian of each layer in reverse order:

$$\frac{\partial L}{\partial x_{k-1}} = \frac{\partial L}{\partial x_k} \frac{\partial x_k}{\partial x_{k-1}}.$$

The Sequential implementation therefore reverses the layer order:

```
def backward(self, dout):
    grad = dout
    for layer in reversed(self.layers):
        grad = layer.backward(grad)
    return grad
```

The model does not need to know the mathematical details of Dense, Conv2D, BatchNorm, or ReLU. It only relies on a consistent local contract: `forward()` transforms activations, and `backward()` transforms the gradient with respect to the layer output into the gradient with respect to its input while filling parameter-gradient buffers when applicable.

12.3 Build lifecycle

Some layer dimensions are known only after the previous layer's output shape is known. A `build()` lifecycle allows each layer to allocate its parameters once the input feature shape is available.

The `built` flag acts as a lifecycle guard. It prevents accidental parameter reinitialization if the model is used again after construction.

A typical model build walks forward through the layers, passing each output shape to the next layer. The important design principle is that parameter creation belongs to the layer that owns those parameters.

12.4 Parameter references

Trainable layers expose parameter/gradient pairs through an interface conceptually like

```
[
    (parameter_array, gradient_array),
    ...
]
```

The returned arrays are references to the actual NumPy arrays stored inside the layers.

This allows an optimizer to update parameters in place without the Sequential model knowing which layer owns them.

It also creates an important reference rule. If an optimizer has retained a reference to a gradient array, a layer should update that gradient array in place:

```
self.dgamma[...] = computed_gradient
```

rather than replacing the array object:

```
self.dgamma = computed_gradient    # may break an existing reference
```

12.5 Train and evaluation mode propagation

The model-level API

```
model.train()
model.eval()
```

propagates the mode to layers whose behavior differs between training and inference.

Most layers are mode-independent. BatchNorm and Dropout are not:

- BatchNorm training uses batch statistics and updates running state; evaluation uses stored running statistics.
- Dropout training samples a random mask; evaluation is identity.

A model-level mode switch avoids forcing the training loop to know which specific layers require mode changes.

12.6 Persistent state and restoration

The project's `get_weights()` / `set_weights()` concept is broader than trainable weights alone. It represents the persistent state required to restore the model's behavior.

For Dense and Conv2D this includes trainable parameters. For BatchNorm it must include

$$\gamma, \beta, \text{running_mean}, \text{running_variance}.$$

This distinction is essential for Early Stopping: restoring only γ and β while leaving later running statistics would not restore the exact inference model that achieved the best validation loss.

13 Optimizers

13.1 Role of an optimizer

Backpropagation computes gradients. It does not decide how parameters are updated. The optimizer receives references to parameter arrays and their corresponding gradient arrays and applies an update rule in place.

For a generic parameter θ with gradient

$$g_t = \frac{\partial L}{\partial \theta_t},$$

different optimizers differ in how they transform g_t before modifying θ_t .

13.2 SGD

Plain stochastic gradient descent uses

$$\theta_{t+1} = \theta_t - \eta g_t$$

where η is the learning rate.

Implementation:

```
for parameter, gradient in self.parameters_grads:
    parameter[...] -= self.learning_rate * gradient
```

There is no per-parameter optimizer state beyond the current parameter itself.

13.3 SGD with Momentum

Momentum maintains a velocity v_t for each parameter. One common convention is

$$v_t = \beta v_{t-1} + (1 - \beta) g_t,$$

followed by

$$\theta_{t+1} = \theta_t - \eta v_t.$$

Another equally common convention omits $(1 - \beta)$ and absorbs the scale into the learning-rate behavior. The implementation and documentation must use one convention consistently.

Conceptually, momentum smooths noisy gradient directions and carries information from previous updates. Therefore each parameter needs its own velocity tensor with the same shape.

```
for parameter, gradient in self.parameters_grads:
    velocity = self.velocity[id(parameter)]
    velocity[...] = beta * velocity + (1.0 - beta) * gradient
    parameter[...] -= learning_rate * velocity
```

Using `id(parameter)` lets optimizer state be associated with the identity of the actual NumPy parameter object rather than with its numerical contents.

13.4 Adam: first and second moments

Adam maintains two state tensors per parameter:

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t, \\ u_t &= \beta_2 u_{t-1} + (1 - \beta_2) g_t^2. \end{aligned}$$

Here m_t is an exponential moving average of gradients, while u_t is an exponential moving average of squared gradients.

At early timesteps both moving averages are biased toward zero because they start from zero. Adam therefore applies bias correction:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{u}_t = \frac{u_t}{1 - \beta_2^t}.$$

The parameter update is

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{u}_t + \epsilon}}.$$

A representative update is:

```
self.t += 1

for parameter, gradient in self.parameters_grads:
    key = id(parameter)
    m = self.m[key]
    u = self.u[key]

    m[...] = self.b1 * m + (1.0 - self.b1) * gradient
    u[...] = self.b2 * u + (1.0 - self.b2) * (gradient ** 2)

    m_hat = m / (1.0 - self.b1 ** self.t)
    u_hat = u / (1.0 - self.b2 ** self.t)

    parameter[...] -= self.learning_rate * (
        m_hat / (np.sqrt(u_hat) + self.epsilon)
    )
```

13.5 Why state is parameter-specific

Each weight tensor has its own optimization history. A Conv2D kernel and a Dense matrix cannot share the same m or u because their shapes and gradient histories differ.

The project associates state with `id(parameter)`. This expresses object identity: two NumPy arrays containing equal values are still different trainable parameters if they are different array objects.

This is also why checking whether a parameter belongs to a selected set of special parameters is naturally an identity question rather than an element-wise array-equality question.

13.6 Classical L2 regularization versus AdamW

Classical L2 regularization modifies the objective:

$$L_{total} = L_{data} + \lambda \sum_j \theta_j^2.$$

Its gradient is

$$\frac{\partial L_{total}}{\partial \theta} = \frac{\partial L_{data}}{\partial \theta} + 2\lambda\theta.$$

With plain SGD this produces an update closely related to weight decay. With adaptive optimizers such as Adam, adding $2\lambda\theta$ to the gradient means the regularization term is also transformed by the adaptive moment machinery.

AdamW instead decouples parameter decay from the data-gradient transformation.

13.7 AdamW update

The Adam portion is computed from the data gradient exactly as before. Separately, selected parameters are decayed:

$$\theta \leftarrow \theta - \eta\lambda\theta - \eta \frac{\hat{m}}{\sqrt{\hat{u}} + \epsilon}.$$

Equivalently,

$$\theta \leftarrow (1 - \eta\lambda)\theta - \eta \text{adam_step}.$$

A representative implementation is:

```
if id(parameter) in self.decayable_ids:
    parameter[...] *= (1.0 - self.learning_rate * self.weight_decay)

parameter[...] -= self.learning_rate * adam_step
```

The important conceptual separation is:

data gradient $\rightarrow$ Adam moments $\rightarrow$ adaptive step
parameter value $\rightarrow$ independent decoupled decay

The decay term does not enter m or u .

13.8 Which parameters are decayable

The final design treats Conv/Dense weight tensors as decayable while excluding common offset/normalization parameters such as

- Dense and Conv biases,
- BatchNorm γ ,
- BatchNorm β .

The optimizer may receive a list of decayable parameter arrays and convert it to

```
self.decayable_ids = {
    id(parameter) for parameter in decayable_parameters
}
```

The membership test then asks whether the current parameter is the exact same NumPy object as one marked for decay.

13.9 In-place updates and reference stability

Because optimizers hold parameter references, updates must mutate the existing arrays:

```
parameter[...] -= update
```

rather than replacing the local variable with a new array object.

This reference-based design keeps the optimizer independent of layer classes while avoiding a heavier `Parameter` abstraction.

14 Early Stopping

14.1 Why Early Stopping is model selection, not backpropagation

Early Stopping does not participate in the computational graph. It observes a validation metric between epochs and decides when further training is no longer useful. It also records the model state corresponding to the best validation result.

For this project, the monitored metric is validation data loss. Smaller is better.

14.2 State

A minimal Early Stopping object needs state such as

- **patience**: how many consecutive non-improving epochs are tolerated,
- **min_delta**: the minimum decrease required to count as a meaningful improvement,
- **best_loss**: best validation loss observed so far,
- a counter of consecutive bad epochs,
- a snapshot of the best persistent model state.

A typical improvement condition is

$$current_loss < best_loss - min_delta.$$

This prevents tiny numerical fluctuations from resetting patience indefinitely.

14.3 Epoch-level logic

Conceptually:

```
if val_loss < best_loss - min_delta:
    best_loss = val_loss
    bad_epochs = 0
    best_state = model.get_weights()
else:
    bad_epochs += 1

if bad_epochs >= patience:
    stop_training = True
```

The exact comparison at the patience boundary should be documented and tested to avoid an off-by-one interpretation. A clear convention is: with **patience=3**, stop after three consecutive validation checks that fail the improvement criterion.

14.4 Why the best state must be copied

The model's parameter arrays continue to change after the best epoch. Therefore Early Stopping cannot merely store references to those arrays. It must snapshot their values, for example with copies inside the model-state interface.

Otherwise the alleged “best weights” would silently keep changing as training continues.

14.5 Restoring BatchNorm correctly

A best-model restore is intended to recreate the inference model from the best validation epoch. For BatchNorm this requires more than γ and β . It also requires the running mean and running variance from that epoch.

Thus the model state should conceptually contain

all persistent state required to reproduce forward inference,

not only arrays updated by the optimizer.

This is an important distinction between

- **trainable parameters**, used by the optimizer, and
- **persistent model state**, used by checkpoint/restore.

14.6 Final test evaluation

The correct final lifecycle is:

1. train while monitoring validation loss;
2. stop when the Early Stopping criterion is met or the maximum epoch count is reached;
3. restore the best model state;
4. switch to evaluation mode;
5. evaluate exactly once on the held-out test set.

The test set should not determine Early Stopping decisions or hyperparameter changes. It is reserved for the final estimate of generalization after model selection.

15 Training and Evaluation Lifecycle

15.1 Mini-batch training

For each epoch, training mode is enabled before processing batches:

```
model.train()

for X_batch, y_batch in create_batches(...):
    logits = model.forward(X_batch)
    loss = criterion.forward(logits, y_batch)

    dlogits = criterion.backward()
    model.backward(dlogits)

    optimizer.update()
```

This order matters:

1. forward caches the current activations/intermediates;
2. loss computes a scalar objective and stores what its backward needs;
3. backward fills gradients for the same forward pass;
4. optimizer updates parameters only after all gradients have been computed.

The training set is shuffled each epoch so that mini-batch composition changes over time.

15.2 Evaluation

Validation and test evaluation use

```
model.eval()
```

followed by forward-only computation. There is no backward call and no optimizer update.

This changes the behavior of Dropout and BatchNorm and also avoids unnecessary gradient work.

15.3 Loss reporting with AdamW

When using decoupled AdamW weight decay, the decay is part of the optimizer update rule rather than an explicit term in the criterion objective. Therefore clean training logs can report

```
train_loss, train_accuracy, val_loss, val_accuracy
```

where the loss is the data/criterion loss.

This makes the logged quantity directly comparable between training and validation and avoids describing AdamW decay as though it were a separate loss term.

15.4 Reproducibility and random number generators

Reproducibility is easier to reason about when randomness is explicit. Random initialization, data shuffling, and Dropout can use NumPy `Generator` objects created from known seeds.

A seed does not guarantee bit-identical behavior across every possible NumPy/platform version, but it provides a reproducible experimental baseline and makes debugging much easier.

16 Numerical Gradient Checking

16.1 Why gradient checking matters

A network can train even when a gradient contains a subtle error. Numerical gradient checking tests the local derivative itself, independently of whether an end-to-end CIFAR run appears plausible.

For a scalar function $L(\theta)$, centered finite differences approximate

$$\frac{\partial L}{\partial \theta_i} \approx \frac{L(\theta_i + h) - L(\theta_i - h)}{2h}.$$

Centered differences have much lower truncation error than one-sided differences for sufficiently small h .

16.2 Converting a tensor output into a scalar test objective

Layer backward methods accept an arbitrary upstream gradient `dout`. To test exactly that backward computation, define the scalar objective

$$L = \sum output \odot dout.$$

In code:

```
L = np.sum(output * dout)
```

Then

$$\frac{\partial L}{\partial output} = dout,$$

so the analytical backward receives exactly the same upstream gradient represented by the finite-difference scalar objective.

16.3 Checking an input gradient

For each small input element x_i :

```
original = x[index]

x[index] = original + h
loss_plus = scalar_loss(layer.forward(x), dout)

x[index] = original - h
loss_minus = scalar_loss(layer.forward(x), dout)

x[index] = original
numerical_dx[index] = (loss_plus - loss_minus) / (2 * h)
```

Compare the resulting numerical tensor with the analytical `din` returned by `backward()`.

16.4 Checking trainable parameters

The same procedure applies to Dense/Conv weights and biases or BatchNorm γ and β . Perturb one parameter element while holding the others fixed, recompute the scalar loss, and compare the finite difference with the corresponding analytical gradient buffer.

For Conv2D, tests should use very small tensors because each parameter/input element requires two forward passes.

16.5 Why float64 is useful for tests

Normal training can use `np.float32` for speed and memory efficiency. Finite differences subtract two nearby floating-point values, which is much more sensitive to rounding error. Therefore gradient-check tests should instantiate layers with `dtype=np.float64`.

This motivates a consistent layer constructor interface with a default training dtype of `float32` but explicit `float64` support for mathematical tests.

16.6 Useful error measures

Absolute error is

$$|g_{\text{analytical}} - g_{\text{numerical}}|.$$

A scale-aware relative measure can be

$$\frac{|g_a - g_n|}{\max(\epsilon, |g_a| + |g_n|)}.$$

The exact threshold depends on the operation, finite-difference step, dtype, and whether the function is near a nondifferentiable point. Very small errors for smooth operations such as Dense and BatchNorm in `float64` provide strong evidence that the implementation follows the derived mathematics.

16.7 Nondifferentiable operations

ReLU at exactly zero and MaxPooling at exact ties are nondifferentiable. Random gradient-check inputs should avoid such boundary cases; otherwise a finite-difference test may disagree with a perfectly reasonable implementation convention.

17 State, Identity, and Mutability in the NumPy Framework

17.1 Why references matter

NumPy arrays are mutable objects. When `model.parameters()` returns an array, the optimizer can hold a reference to the same object stored by the layer. Therefore

```
parameter[...] -= update
```

mutates the layer's real parameter tensor.

No explicit “copy back into the layer” step is required.

17.2 Identity is different from equality

Two arrays can contain equal values without representing the same parameter object. Optimizer bookkeeping asks questions such as

“Is this exact parameter one of the arrays that should receive weight decay?”

not

“Does this array currently contain values equal to some decayable array?”

Therefore object identity, represented by `id(parameter)`, is appropriate for optimizer state maps and decayable-parameter sets.

17.3 Gradient-buffer identity

The same reasoning applies to gradient buffers. If an optimizer receives a reference to `self.dweights`, then later replacing

```
self.dweights = new_array
```

creates a different object and can leave the optimizer pointing to the old one. Mutating instead with

```
self.dweights[...] = new_values
```

preserves identity.

This is a low-level framework detail, but it is an important consequence of choosing a lightweight array-reference design instead of a dedicated Parameter class.

18 Putting the Layers Together: One Backward Pass Through the Network

Consider a simplified chain

$$X \rightarrow \text{Conv} \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{Pool} \rightarrow \text{Flatten} \rightarrow \text{Dense} \rightarrow \text{Loss}.$$

The loss first produces $dL/d\text{logits}$. Then:

1. Dense backward computes its dW , db , and sends dL/dX_{dense} to Flatten.
2. Flatten restores the convolutional activation shape without changing gradient values.
3. MaxPooling routes each upstream gradient to the stored winning input position, accumulating if pooled windows overlap.
4. ReLU masks gradients according to which pre-ReLU activations were positive.

5. BatchNorm computes $d\gamma$, $d\beta$, and propagates through normalization, variance, and mean, adding the multiple graph paths into each input.
6. Conv2D computes db , accumulates shared-weight contributions into dW , and computes dX by first summing over filters per window occurrence and then scatter-adding overlapping occurrences back into input coordinates.

The Sequential container only orchestrates this reverse order. Every mathematical detail remains local to the layer that owns the corresponding forward transformation.

19 A Worked Shape Trace for the CIFAR-10 Architecture

For batch size 32, an example architecture is:

Operation	Output shape	Comment
Input	(32, 32, 32, 3)	NHWC RGB batch
Conv2D, 32 filters	(32, 32, 32, 32)	3×3 , stride 1, padding 1
BatchNorm + ReLU	(32, 32, 32, 32)	channel-wise BN
Conv2D, 32 filters	(32, 32, 32, 32)	preserves spatial size
BatchNorm + ReLU	(32, 32, 32, 32)	
MaxPool	(32, 16, 16, 32)	spatial downsampling
Conv2D, 64 filters	(32, 16, 16, 64)	
BatchNorm + ReLU	(32, 16, 16, 64)	
Conv2D, 64 filters	(32, 16, 16, 64)	
BatchNorm + ReLU	(32, 16, 16, 64)	
MaxPool	(32, 8, 8, 64)	
Flatten	(32, 4096)	$8 \cdot 8 \cdot 64 = 4096$
Dense	(32, 256)	hidden representation
BatchNorm + ReLU	(32, 256)	feature-wise BN
Dropout	(32, 256)	training only
Dense	(32, 10)	logits

The important architectural interface is that Conv2D and MaxPooling use four-dimensional NHWC tensors, while Flatten converts the representation into the two-dimensional batch-feature layout expected by Dense.

20 Testing Strategy

The project benefits from two complementary categories of tests.

20.1 Behavioral/API tests

These test software contracts rather than derivatives. Examples include:

- batch creation and shuffling behavior,
- train/validation split behavior,
- optimizer construction,
- Dropout evaluation identity,
- BatchNorm running statistics changing in training but not evaluation,
- `get_weights()` / `set_weights()` restoring persistent BatchNorm state,

- Early Stopping patience/min-delta semantics,
- AdamW not decaying biases or BatchNorm affine parameters,
- learning rate and optimizer constructor validation.

20.2 Mathematical tests

Numerical finite-difference tests should cover at least:

- Dense: dX , dW , db ,
- Conv2D: dX , dW , db on tiny tensors, including stride/padding cases,
- BatchNorm: dX , $d\gamma$, $d\beta$ using float64,
- optionally smooth local checks for other differentiable operations.

An end-to-end CIFAR-10 training run is valuable integration verification, but it is not a substitute for unit-level mathematical tests. The two answer different questions:

“Can the whole system learn?” versus “Is this derivative actually the derivative of this forward operation?”

21 Design Principles Reflected by the Framework

21.1 Keep abstractions proportional to the project

The framework intentionally avoids introducing a large object hierarchy unless it solves an existing problem. For example, array references plus a small `parameters()` interface are sufficient for optimizer updates, so a full Parameter abstraction is not currently necessary.

Likewise, a simple Sequential model is enough for a chain architecture. A general dynamic computation graph would add complexity without improving the educational goal of the project.

21.2 Make state ownership explicit

Each responsibility has a natural owner:

- layers own parameters, gradients, and forward caches;
- the Sequential model owns layer ordering and model mode propagation;
- the loss owns the objective and its derivative with respect to logits;
- the optimizer owns optimization state such as momentum/Adam moments and the current learning rate;
- Early Stopping owns validation-history state and the best-model snapshot.

Clear ownership reduces hidden coupling between parts of the framework.

21.3 Prefer explainable vectorization

Vectorization is valuable because NumPy executes array operations in optimized compiled code, but the final expression should remain derivable from the scalar definition.

The strongest examples in the project are:

- Dense dW : an explicit sum of per-sample outer products becomes `dout.T @ X`;
- Conv2D dW : a sum across every use of a shared kernel weight becomes `flat_dout.T @ flat_windows`;

- Conv2D per-window dX : a sum over filters becomes `flat_dout @ flat_weights`;
- Conv2D final dX : overlapping window occurrences are merged with scatter-add;
- MaxPooling backward: explicit batch/channel routing becomes broadcasting plus advanced indexing;
- BatchNorm: a complicated compact derivative is replaced by a sequence of transparent vectorized local derivatives.

22 Compact Reference: Forward and Backward Equations

22.1 Dense

$$Z = XW^T + b,$$

$$dX = dout W, \quad dW = dout^T X, \quad db = \sum_{batch} dout.$$

22.2 ReLU

$$y = \max(0, x), \quad dx = dout \mathbf{1}[x > 0].$$

22.3 Conv2D

$$z_{b,h,w,f} = \sum_{k_r, k_c, c} X_{b,hs+k_r,ws+k_c,c}^{(p)} W_{f,kr,kc,c} + b_f.$$

$$db_f = \sum_{b,h,w} dout_{b,h,w,f}.$$

$$dW_{f,kr,kc,c} = \sum_{b,h,w} dout_{b,h,w,f} X_{b,hs+kr,ws+kc,c}^{(p)}.$$

$$dX_{b,i,j,c}^{(p)} = \sum_{\text{window occurrences mapping to } (i,j,c)} \sum_f dout W.$$

22.4 MaxPooling

$$y = \max(window),$$

$$dx_j = \begin{cases} dout, & j = \operatorname{argmax}(window), \\ 0, & \text{otherwise.} \end{cases}$$

22.5 BatchNorm

$$\mu = \operatorname{mean}(x), \quad \tilde{x} = x - \mu,$$

$$\sigma^2 = \operatorname{mean}(\tilde{x}^2), \quad r = (\sigma^2 + \epsilon)^{-1/2},$$

$$\hat{x} = \tilde{x}r, \quad y = \gamma\hat{x} + \beta.$$

The backward is most safely implemented by reversing exactly these operations.

22.6 Dropout

Training:

$$y = x \frac{m}{1-p}, \quad dx = dout \frac{m}{1-p}.$$

Evaluation:

$$y = x.$$

22.7 Softmax Cross-Entropy

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}, \quad L = -\frac{1}{B} \sum_b \log p_{b, y_b},$$

$$\boxed{dZ = \frac{P - Y}{B}}.$$

A Appendix A: Conv2D Backward as Index Mappings

This appendix collects the two index mappings that make Conv2D backward easier to reconstruct from memory.

A.1 Window occurrence to padded input

Every extracted window element satisfies

$$window[b, h, w, k_r, k_c, c] = padded_input[b, h_s + k_r, w_s + k_c, c].$$

Therefore the backward scatter uses the inverse interpretation:

A gradient stored at window coordinate (b, h, w, k_r, k_c, c) must be added to padded-input coordinate $(b, h_s + k_r, w_s + k_c, c)$.

A.2 Flattened matrix indices

For matrix multiplication, define

$$p \leftrightarrow (b, h, w), \quad q \leftrightarrow (k_r, k_c, c).$$

Then

$$flat_windows[p, q] = window[b, h, w, k_r, k_c, c],$$

$$flat_dout[p, f] = dout[b, h, w, f],$$

$$flat_weights[f, q] = W[f, k_r, k_c, c].$$

This makes the two core matrix multiplications almost self-deriving:

$$\underbrace{flat_dout^T}_{F \times P} \underbrace{flat_windows}_{P \times Q} \rightarrow \underbrace{dW_{flat}}_{F \times Q},$$

$$\underbrace{flat_dout}_{P \times F} \underbrace{flat_weights}_{F \times Q} \rightarrow \underbrace{dwindows_{flat}}_{P \times Q}.$$

The first sums over all window uses of each weight. The second sums over all filters through which one window element affects the loss.

B Appendix B: BatchNorm Computational Graph Checklist

When re-deriving BatchNorm, write the forward as:

1. $\mu = mean(x)$,
2. $x_c = x - \mu$,
3. $v = mean(x_c^2)$,
4. $r = (v + \epsilon)^{-1/2}$,
5. $\hat{x} = x_c r$,
6. $y = \gamma \hat{x} + \beta$.

Then reverse the list. Whenever one forward variable fans out to multiple later operations, add the gradient contributions when they meet again. This procedure is more robust than trying to remember a final simplified dx formula.

C Appendix C: Suggested Repository Documentation Split

A clean repository can keep the README concise while linking to this technical document.

The README can focus on:

- project purpose and learning goals,
- implemented layers and optimizers,
- example model architecture,
- installation and run instructions,
- project structure,
- test instructions,
- final CIFAR-10 results,
- a short link to these mathematical/implementation notes.

This document can remain the long-form explanation for readers who want to inspect the derivations and implementation reasoning in depth.

D Appendix D: Final Mental Models

Dense dW . One weight is reused by every sample. Compute one contribution $dout \cdot x$ per sample and sum them. Matrix multiplication performs all those sums at once.

Conv2D dW . One kernel weight is reused at every batch/spatial window. Its gradient is the sum of $dout \cdot$ corresponding-window-element over every use. `flat_dout.T @ flat_windows` is exactly that sum.

Conv2D dX . First compute the gradient of every input occurrence inside every window by summing through all filters. Then map window occurrences back to real input coordinates and add occurrences that refer to the same pixel.

MaxPooling dX . Remember the forward winner. Backward routes the upstream gradient only to that winner. Accumulate if overlapping windows route to the same input.

BatchNorm dX . Do not memorize the compact expression. Reverse the graph. An input influences normalization directly and through shared mean/variance statistics, so multiple paths must be summed.

Sequential. Forward is function composition from first layer to last. Backward is the chain rule applied in the reverse order.

AdamW. The data gradient goes through Adam's moment normalization. Weight decay is a separate parameter update and does not enter the moment estimates.

Early Stopping. Validation decides which persistent model state is best. Restore that state before final test evaluation; inference state such as BatchNorm running statistics is part of the model snapshot.

Closing Note

The main achievement of a from-scratch neural-network framework is not reproducing a familiar API. It is developing the ability to move between scalar mathematics, computational graphs, tensor shapes, explicit loops, and vectorized array operations without losing track of what each number represents.

Whenever a vectorized line becomes difficult to understand, the reliable way back is:

1. name one scalar output;
2. ask which scalar inputs/parameters influence it;
3. write the local derivative;
4. identify every path from the variable to the loss;
5. sum those path contributions;
6. write the explicit loops that perform the sum;
7. only then ask which loops NumPy can replace with reductions, matrix multiplication, broadcasting, reshaping, or scatter-add.

That reasoning process is the common thread connecting every backward pass in this project.